

Lecture 12:

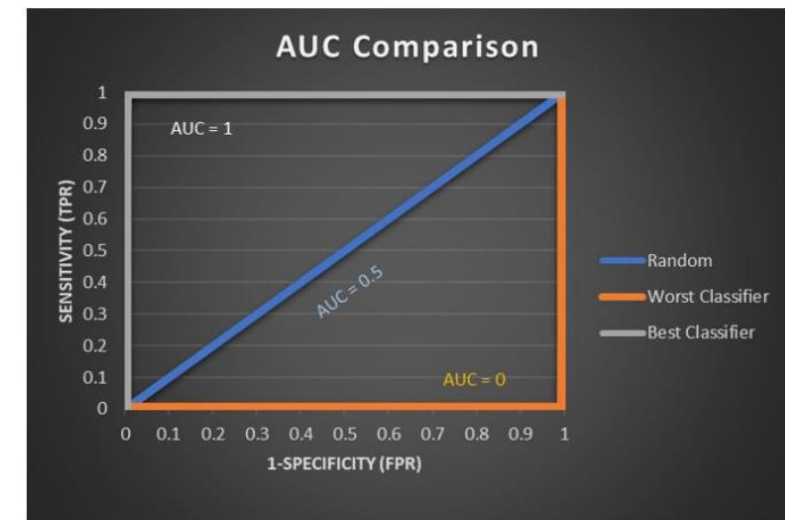
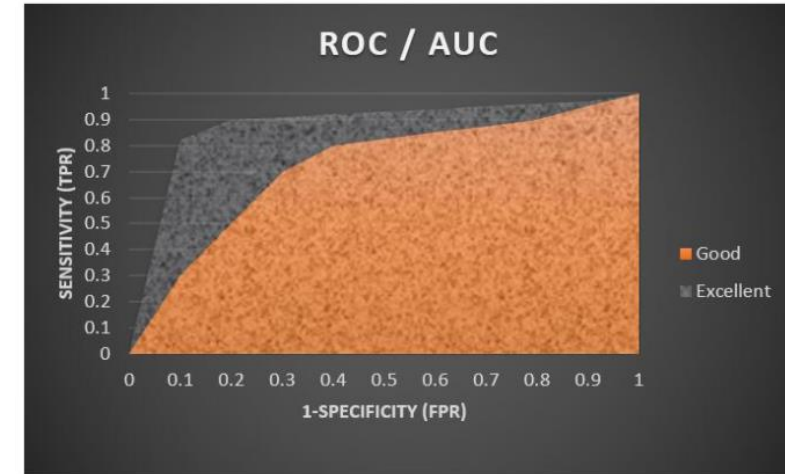
Classification Evaluation Metrics and Clustering Algorithms

ML/AI for Engineering Applications
ENV 6932

Sara Behdad
Fall 2025

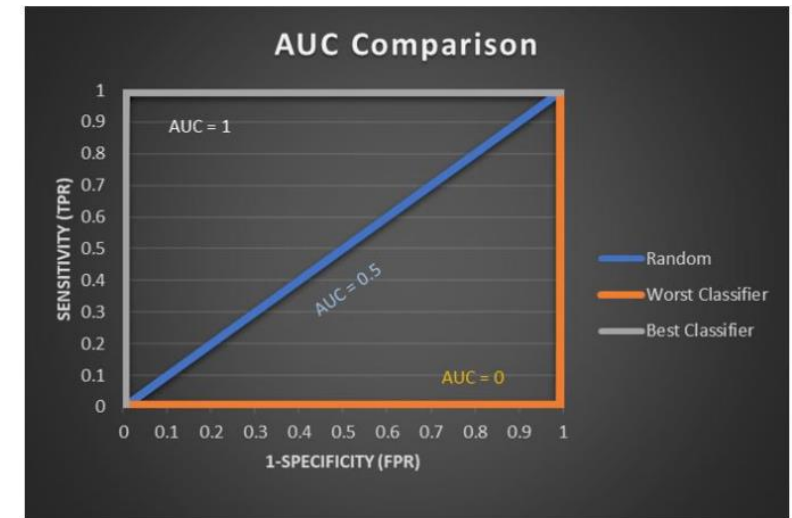
AUC - Area Under the Curve

- AUC offers an aggregate measure of performance of a model **across all possible thresholds**.
- AUC shows a measure of separability.
- It shows the capability of the model in distinguishing between classes.
- Higher the AUC, better the model is at predicting the probability of class YES higher than the probability of class NO.
- AUC ranges from 0 to 1. A model whose predictions are 100% right has an AUC of 1.0; one whose predictions are 100% wrong has an AUC of 0.0.



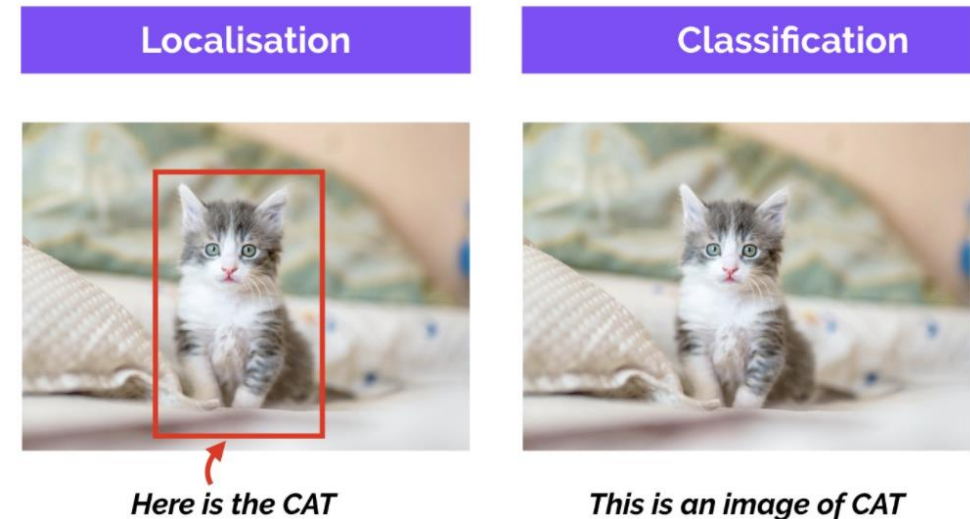
Is the random model the worst possible model?

- The random classifier predicts an observation as class YES or NO at random. The AUC would be 0.5 and $TPR = FPR$ at all thresholds.
- The worst possible model would be a model that predict all the class YES obs as class NO and class NO obs as class YES. The AUC of such a model would be 0.



mAP (mean Average Precision)

- Commonly used in computer vision
- A popular evaluation metric for object detection (i.e. localization and classification).
- Two different tasks to measure
 - **Localization**: the location of an instance (e.g., bounding box coordinates)
 - **Classification** determines the label (e.g., a dog or cat).



Evaluating Object Detectors Using mAP

- In a typical dataset there would be many classes with non-uniform distribution (e.g., many more dogs than trees).
- A simple accuracy-based metric does not work. Also, we need to assess the risk of misclassifications.
- Thus, there is a need to attach a “confidence score” or **model score** with each bounding box detected and to calculate the model at various levels of confidence.
- So, the Average Precision (AP) is introduced.

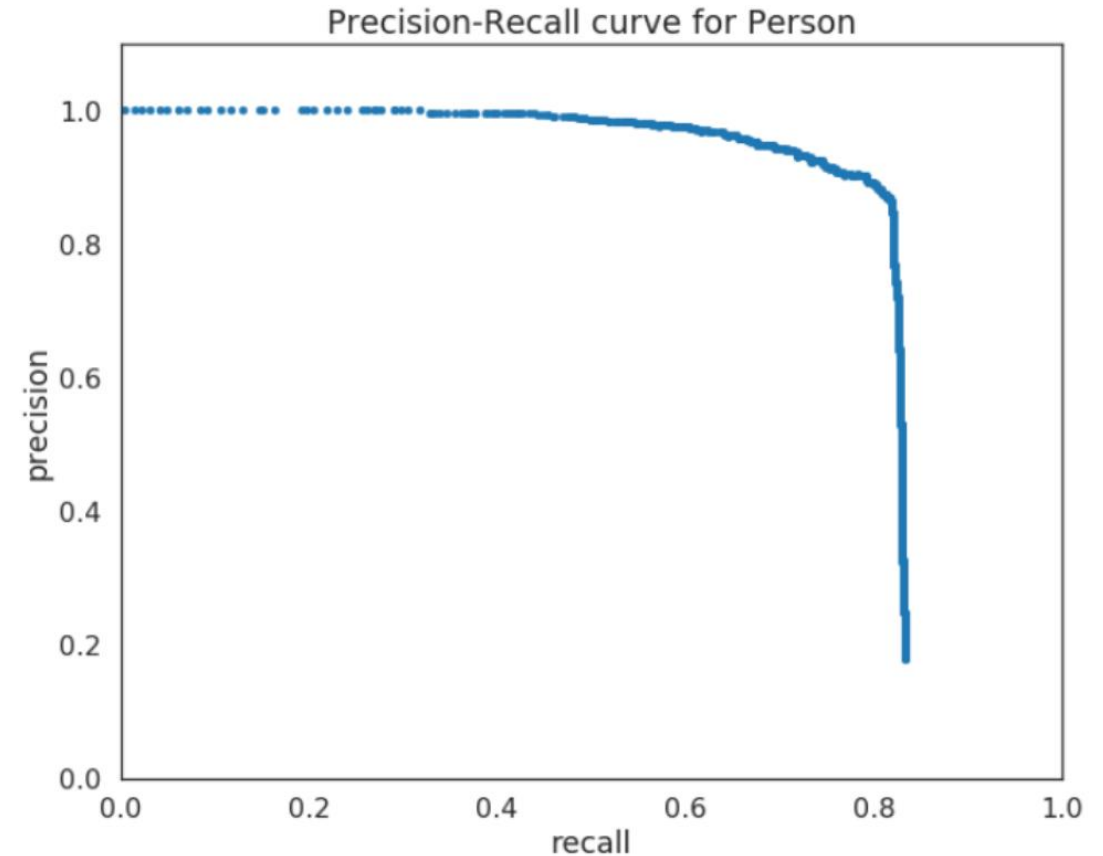
Evaluating Object Detectors Using mAP

- What is AP or mAP?
- First, we need to understand the Precision and Recall.
- **Precision:** how accurate the predictions are (% of your predictions that are correct).
- **Recall:** measures how good you find all the positives.

$$\text{Precision} = \frac{TP}{TP + FP} \qquad \text{Recall} = \frac{TP}{TP + FN}$$

Calculating mAP for Classification

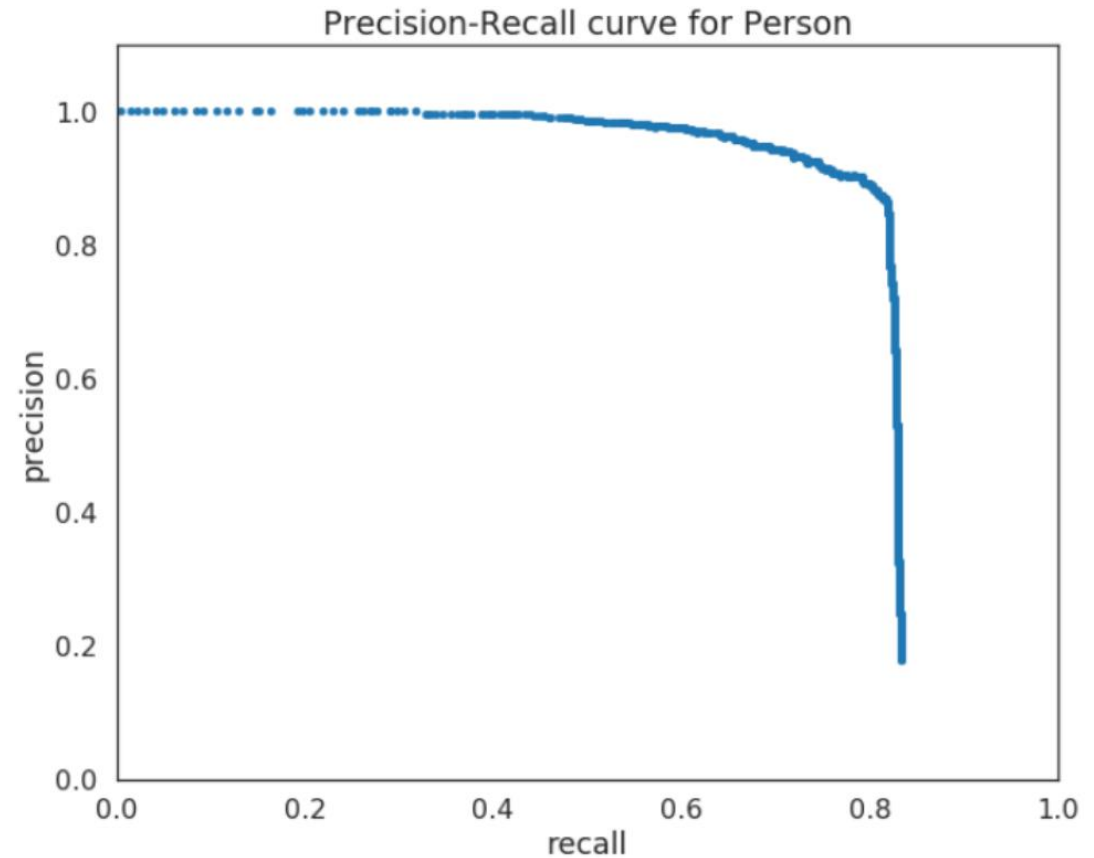
- How to find a single value that summarizes the shape of the precision-recall curve??
- For a specific class (e.g., cat) draw the precision-recall curve, by varying the model score threshold that determines what is counted as a positive detection of the class.
- Example: each point on the curve represents a different score threshold:



Calculating mAP for Classification

- Finally, calculate the AP score by taking the average value of the precision among all recall values.
- The AP score is determined as the mean of precision at the set of 11 equally spaced recall values, $Recall_i = [0, 0.1, 0.2, \dots, 1.0]$.
- by segmenting the recalls evenly to 11 parts: $\{0, 0.1, 0.2, \dots, 0.9, 1\}$

$$AP = \frac{1}{11} \sum_{Recall_i} Precision(Recall_i)$$



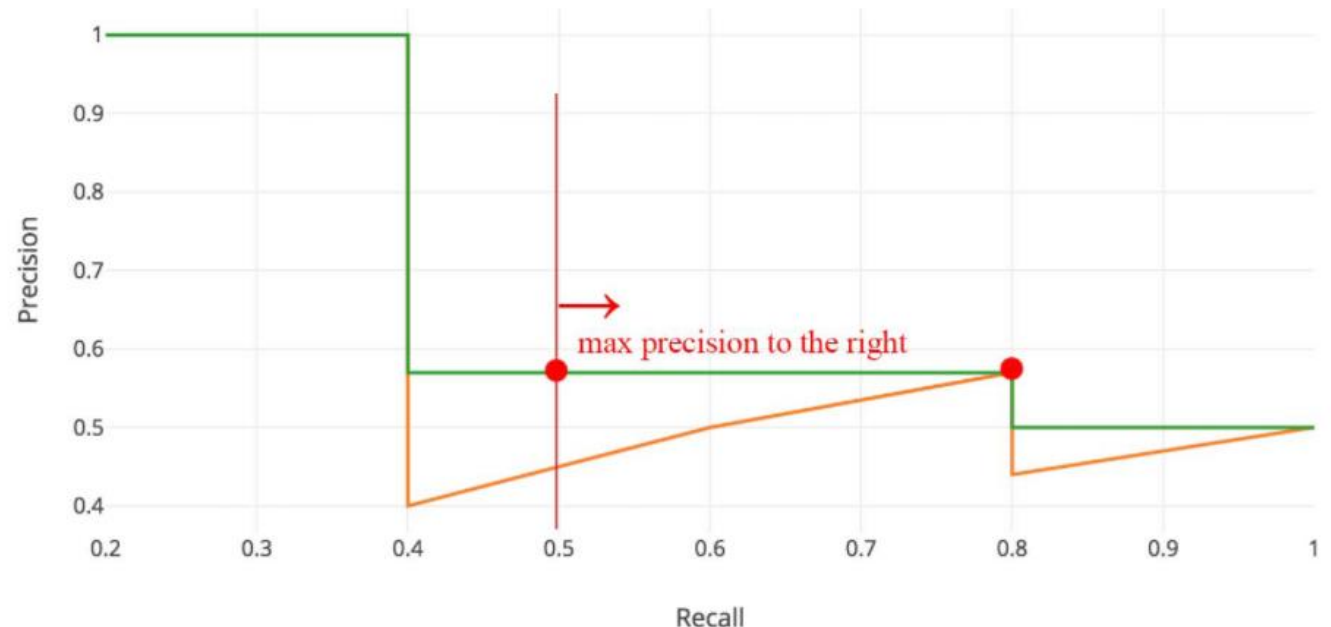
Calculating mAP for Classification

- The AP summarizes the shape of the precision-recall curve.
- The area under the PR curve is called **Average Precision (AP)**
- The mean of precision values at a set of 11 equally spaced recall levels [0,0.1,...,1] (0 to 1 at step size of 0.1).

$$AP = \frac{1}{11} \sum_{r \in (0,0.1,\dots,1)} p_{interp}(r)$$

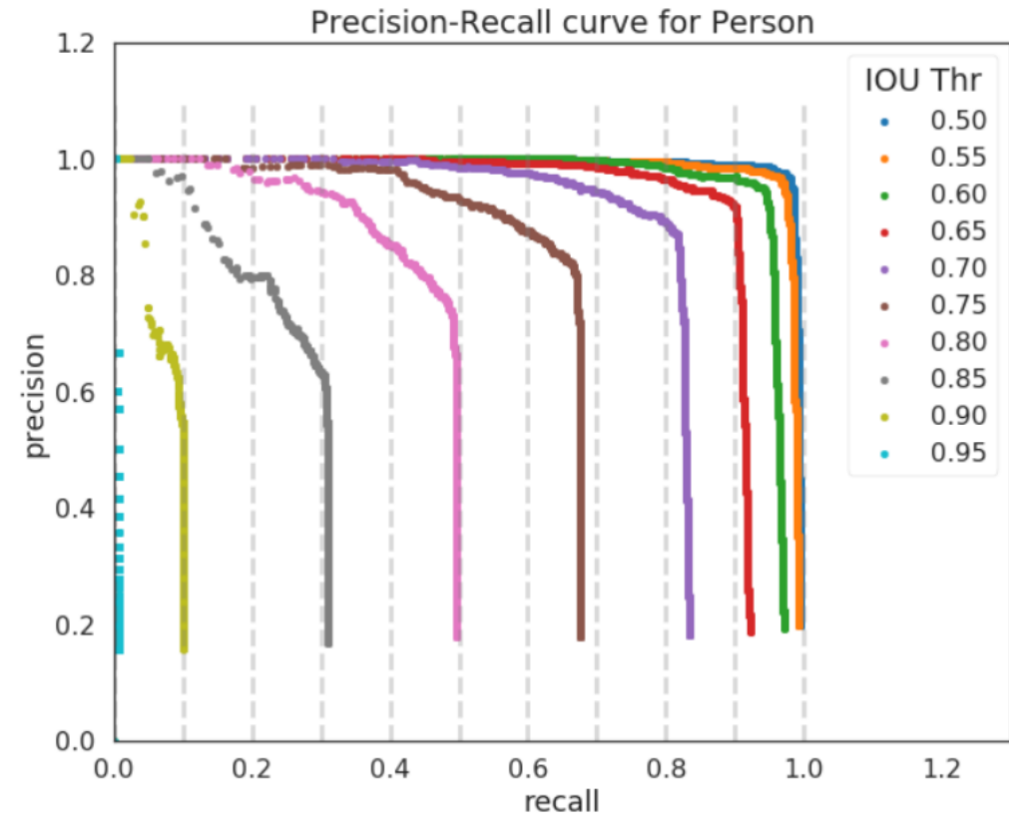
$$p_{interp}(r) = \max_{\tilde{r}:\tilde{r} \geq r} p(\tilde{r})$$

Take the max precision to the right at 11 equally spaced recall points [0: 0.1: 1], and take their mean.



Calculating mAP for Localization

- To evaluate how well the model predicted the location of the object, often **the Intersection over Union threshold (IoU)** is used.
- The object detections are true or false depending upon the IoU threshold.
- Calculate Precision and Recall for different IoU thresholds and drawn the precision-recall curve (e.g., from 0.5 to 0.95 in steps of 0.05).



Calculating mAP for Localization

- **The mAP score is calculated by taking the mean AP over all classes and/or over all IoU thresholds.**
- Examples:
 - PASCAL VOC2007 challenge only 1 IoU threshold (0.5), so the mAP was averaged over all 20 classes.
 - In COCO 2017 challenge, the mAP was averaged over all 80 object categories and all 10 IoU thresholds.

- Example using COCO (**Common Objects in Context**) dataset

Average Precision (AP):

AP % AP at IoU=.50:.05:.95 (primary challenge metric)
AP_{IoU=.50} % AP at IoU=.50 (PASCAL VOC metric)
AP_{IoU=.75} % AP at IoU=.75 (strict metric)

AP Across Scales:

AP_{small} % AP for small objects: area < 32²
AP_{medium} % AP for medium objects: 32² < area < 96²
AP_{large} % AP for large objects: area > 96²

Average Recall (AR):

AR_{max=1} % AR given 1 detection per image
AR_{max=10} % AR given 10 detections per image
AR_{max=100} % AR given 100 detections per image

AR Across Scales:

AR_{small} % AR for small objects: area < 32²
AR_{medium} % AR for medium objects: 32² < area < 96²
AR_{large} % AR for large objects: area > 96²



- AP@.75 means the AP with IoU=0.75

	backbone	AP	AP ₅₀	AP ₇₅	AP _S	AP _M	AP _L
<i>Two-stage methods</i>							
Faster R-CNN+++ [3]	ResNet-101-C4	34.9	55.7	37.4	15.6	38.7	50.9
Faster R-CNN w FPN [6]	ResNet-101-FPN	36.2	59.1	39.0	18.2	39.0	48.2
Faster R-CNN by G-RMI [4]	Inception-ResNet-v2 [19]	34.7	55.5	36.7	13.5	38.1	52.0
Faster R-CNN w TDM [18]	Inception-ResNet-v2-TDM	36.8	57.7	39.2	16.2	39.8	52.1
<i>One-stage methods</i>							
YOLOv2 [13]	DarkNet-19 [13]	21.6	44.0	19.2	5.0	22.4	35.5
SSD513 [9, 2]	ResNet-101-SSD	31.2	50.4	33.3	10.2	34.5	49.8
DSSD513 [2]	ResNet-101-DSSD	33.2	53.3	35.2	13.0	35.4	51.1
RetinaNet [7]	ResNet-101-FPN	39.1	59.1	42.3	21.8	42.7	50.2
RetinaNet [7]	ResNeXt-101-FPN	40.8	61.1	44.1	24.1	44.2	51.2
YOLOv3 608 × 608	Darknet-53	33.0	57.9	34.4	18.3	35.4	41.9

COCO for YOLOv3

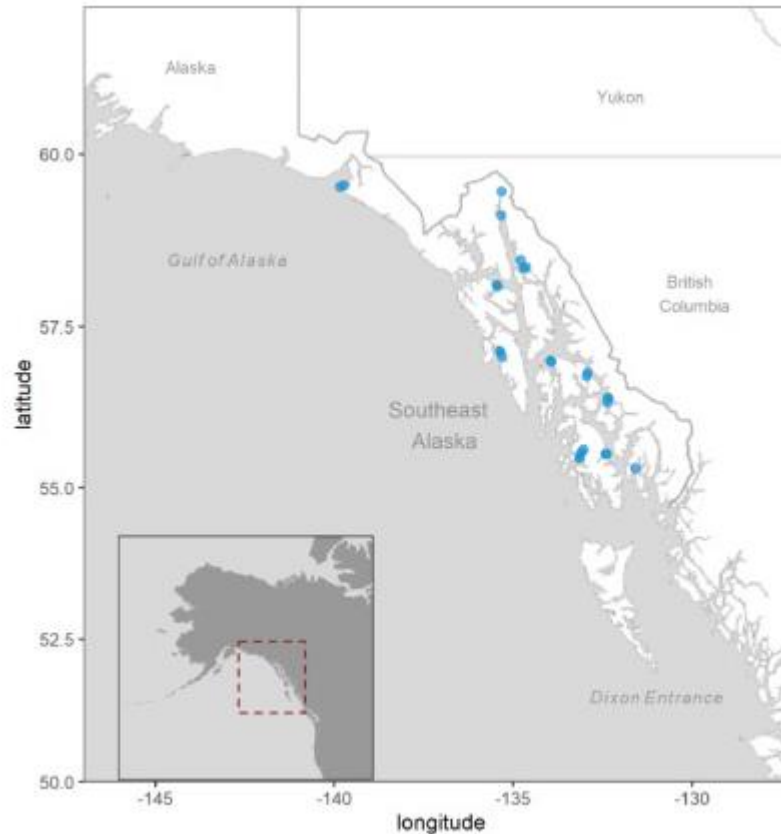
Example: Random forest to determine environmental drivers and forecast PSTs in Alaska

- The concentrations of paralytic shellfish toxins (PSTs) regularly exceed FDA thresholds in Southeast Alaska.
- **Response factor:** PSTs in blue mussels on daily timescales.
- **Key variables:** Salinity, temperature, freshwater discharge and air temperature.



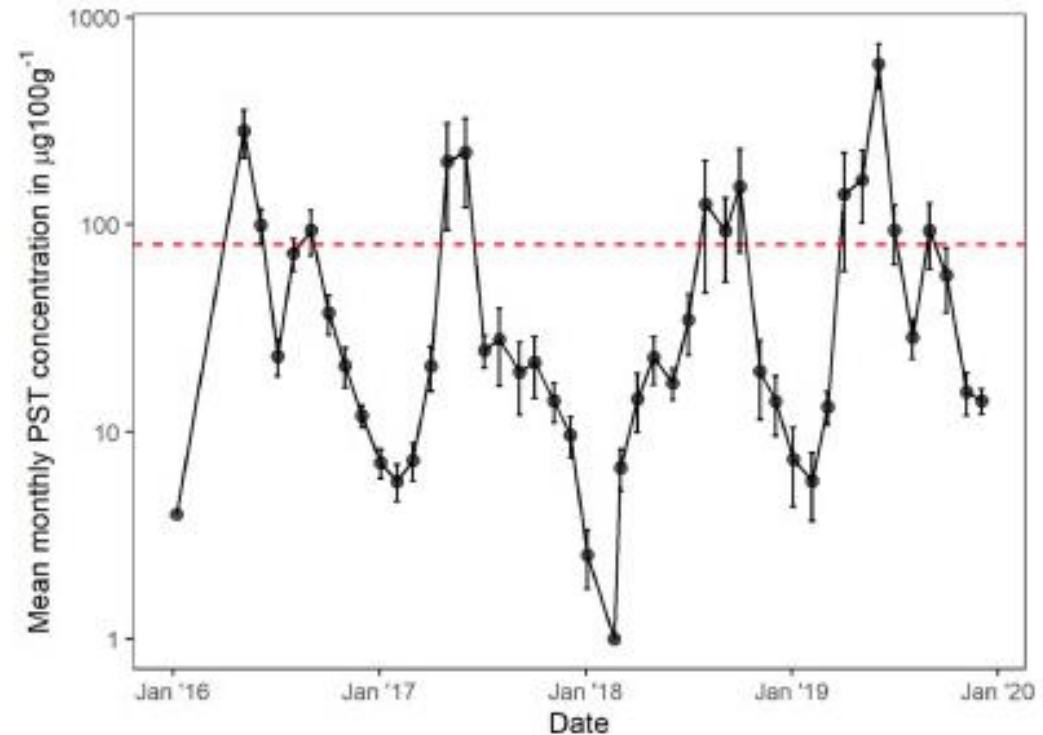
Example: Random forest to determine environmental drivers and forecast PSTs in Alaska

- During each shellfish sampling: 20-100 blue mussel samples are collected and shipped to the Research Lab for analysis.
- tested 986 blue mussel samples from 40 sample sites in Southeast Alaska



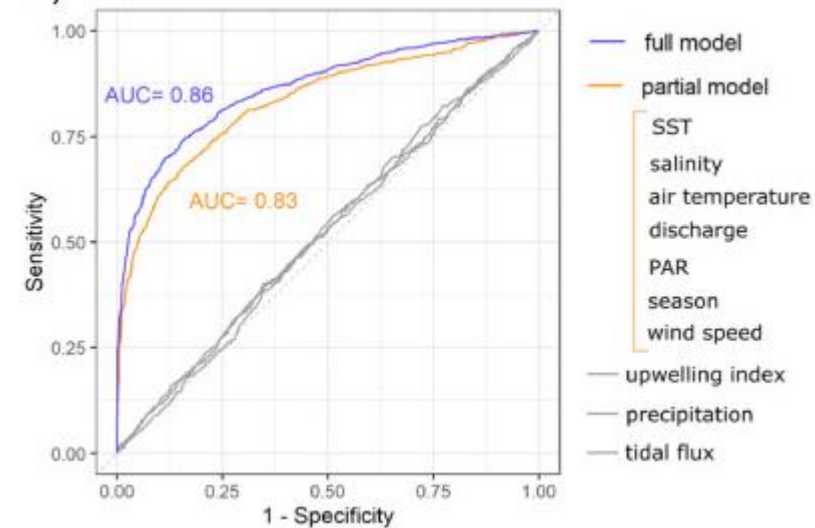
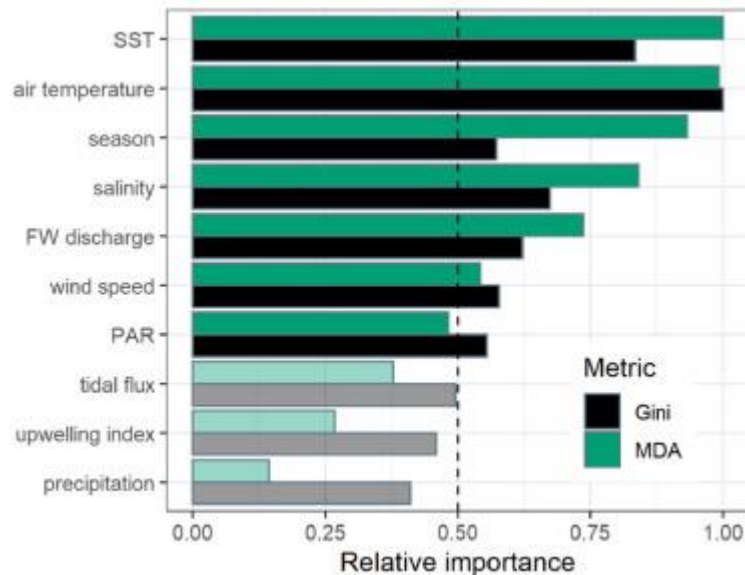
Example: Random forest to determine environmental drivers and forecast PSTs in Alaska

- Mean monthly PST concentrations for samples collected since 2016.
- The dashed red line indicates the $80 \mu\text{g } 100 \text{ g}^{-1}$ action threshold set by the US FDA.



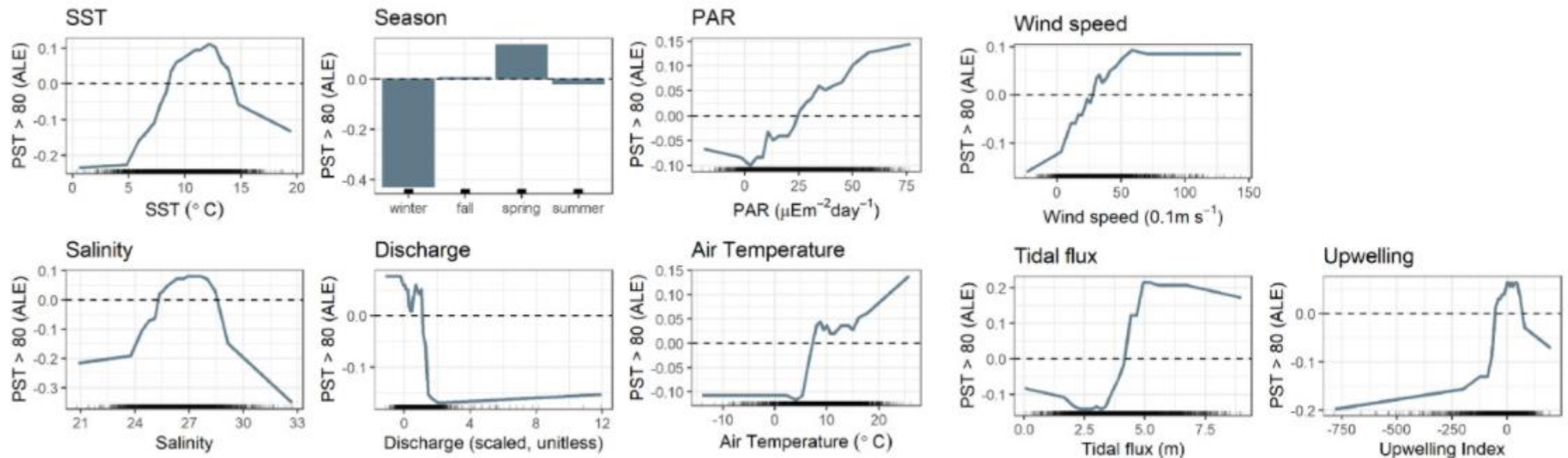
Example: Random forest to determine environmental drivers and forecast PSTs in Alaska

- Variable importance measures MDA and Gini showed that SST, air temperature, season, and salinity were among the most important features of the model.



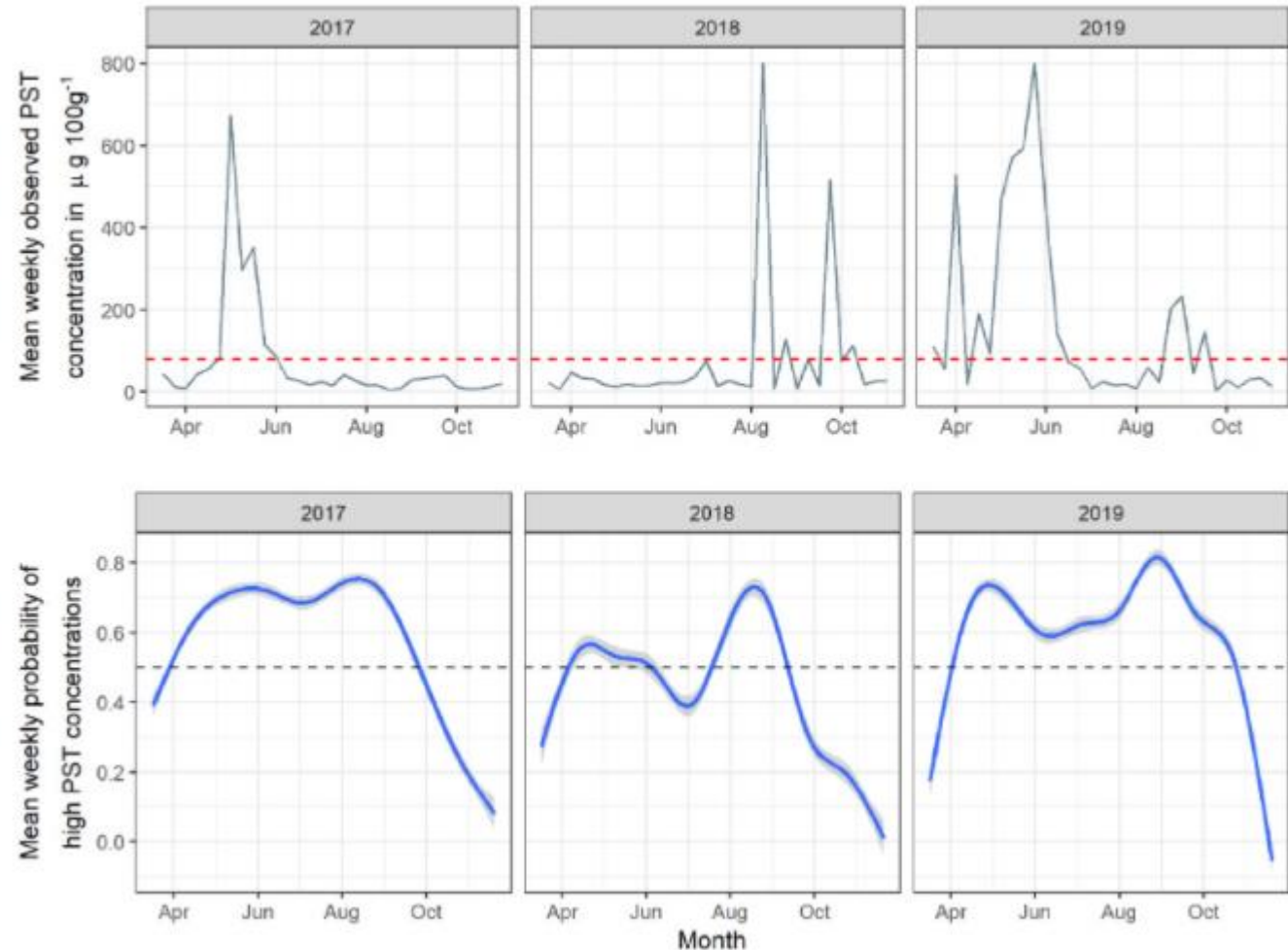
Example: Random forest to determine environmental drivers and forecast PSTs in Alaska

- Accumulated local effect (ALE) plots for each variable in the full model. The y-axes represent the effect of that variable on predictions of concentrations $> 80 \mu\text{g } 100 \text{ g}^{-1}$ (positive values) and predictions of concentrations $< 80 \mu\text{g } 100 \text{ g}^{-1}$ (negative values).



Example: Random forest to determine environmental drivers and forecast PSTs in Alaska

- Weekly average PST concentrations during April–November in 2017, 2018, and 2019.
- Red dashed: the FDA threshold of $80 \mu\text{g } 100 \text{ g}^{-1}$. b)
- Probabilities close to 1 indicate high confidence of PST concentrations above $80 \mu\text{g } 100 \text{ g}^{-1}$, while probabilities near 0 indicate high confidence of PST concentrations less than $80 \mu\text{g } 100 \text{ g}^{-1}$.



References

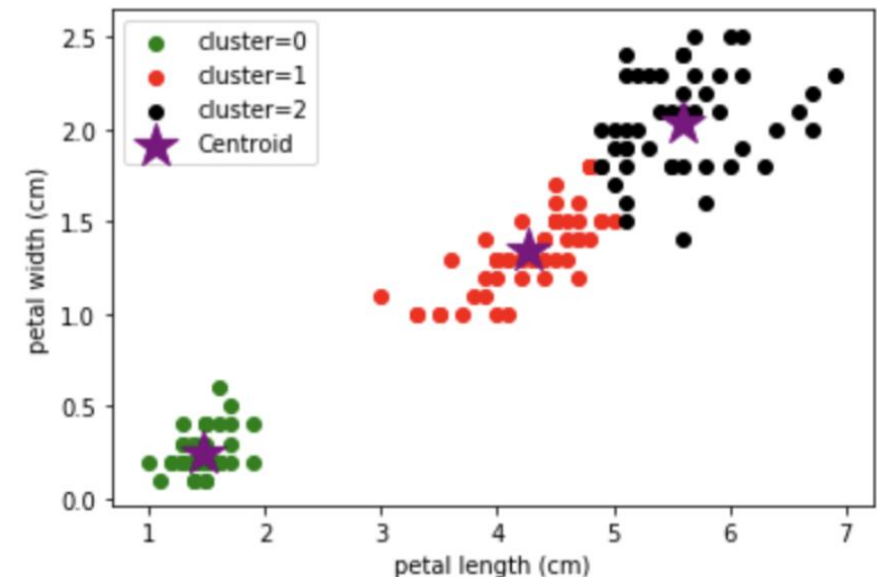
- <https://www.analyticsvidhya.com/blog/2020/06/4-ways-split-decision-tree/>
- <https://medium.com/swlh/decision-tree-classification-de64fc4d5aac>
- <https://towardsdatascience.com/entropy-how-decision-trees-make-decisions-2946b9c18c8>

Outline

- **Clustering Algorithms**
 - K-Means
 - Mean-shift
 - Density-Based Spatial Clustering of Applications with Noise (DBSCAN)
 - Agglomerative Hierarchical Clustering

Clustering Algorithms

- Clustering algorithms or Cluster analysis are unsupervised machine learning algorithms.
- They **automatically discover natural grouping in data**.
- Different from supervised learning, clustering only interprets the input data and find natural groups in feature space.



Clustering Algorithms

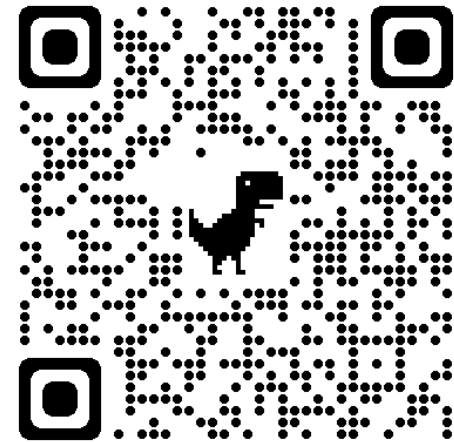
- K-Means Clustering
- Mean-Shift Clustering
- Density-Based Spatial Clustering of Applications with Noise (DBSCAN)
- Expectation–Maximization (EM) Clustering using Gaussian Mixture Models (GMM)
- Agglomerative Hierarchical Clustering

K-Means

- K-Means discovers underlying patterns in the dataset and groups similar data points together.
- It partitions n objects into k clusters in which each object belongs to the cluster with the nearest mean.
- It creates **sphere-shaped** clusters.
- K-Means is suitable for **medium and large** datasets.

K-Means

- K : The number of centroids we need in the dataset.
- Centroid: the imaginary or real location demonstrating the center of a cluster.



QR code for K-means example

K-Means

- The best k leading to the greatest separation (distance) is not known as a priori and must be computed from the data.
- The objective function:

The diagram shows the objective function J for K-Means clustering. The equation is $J = \sum_{j=1}^k \sum_{i=1}^n \|x_i^{(j)} - c_j\|^2$. Annotations include: an arrow from 'number of clusters' to k ; an arrow from 'number of cases' to n ; an arrow from 'case i ' to $x_i^{(j)}$; an arrow from 'centroid for cluster j ' to c_j ; a bracket under the distance term labeled 'Distance function'; and an arrow from 'objective function' to J .

$$\text{objective function} \leftarrow J = \sum_{j=1}^k \sum_{i=1}^n \underbrace{\|x_i^{(j)} - c_j\|^2}_{\text{Distance function}}$$

K-Means Algorithm Steps

- **Step 1:** *choose a value for the number of groups/clusters/classes, K*
- **Step 2:** randomly initialize K center points.
- **Step 3:** assign each point to a closest cluster by computing the distance between that point and each group center.
- **Step 4:** after all points are assigned to clusters, calculate the mean of each cluster and select the mean of each cluster as the new center point for that cluster (centroid), and go to Step 3.
- **Step 5:** Repeat Steps 3 and 4 for a predefined number of iterations or until the group centers don't change much between iterations.

Example: K-Means

- Step 1: Select K, let's consider $K=2$



(a)

Example: K-Means

- Step 2: randomly initialize K center points.



(a)



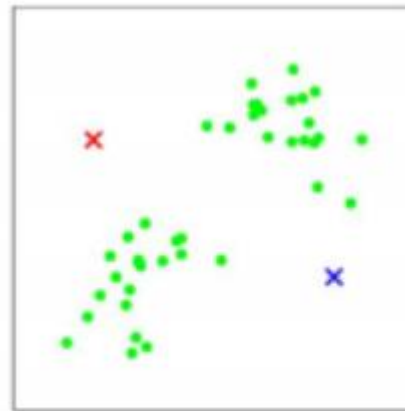
(b)

Example: K-Means

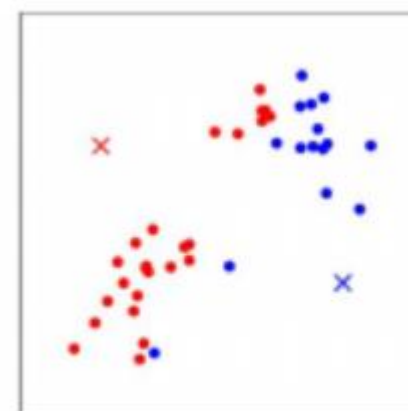
- Step 3: assign each point to a closest cluster by computing the distance between that point and each group center.



(a)



(b)



(c)

Example: K-Means

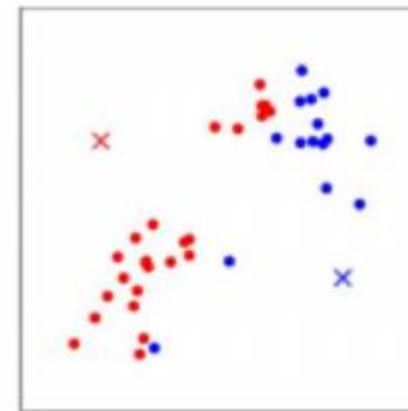
- Step 4: calculate the mean of each cluster and select the mean as the new centroid



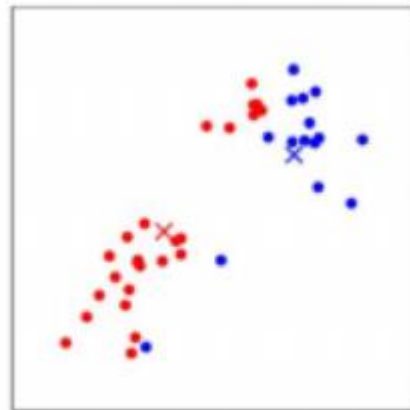
(a)



(b)



(c)



(d)

Example: K-Means

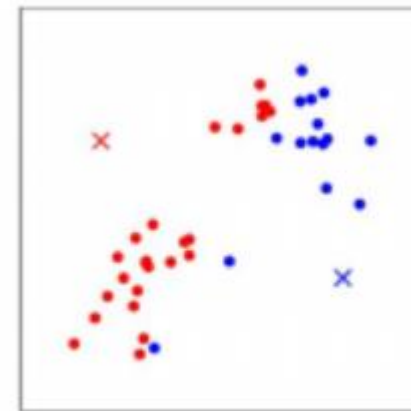
- Step 5: Repeat Steps 3 and 4 and reassign data points to clusters.



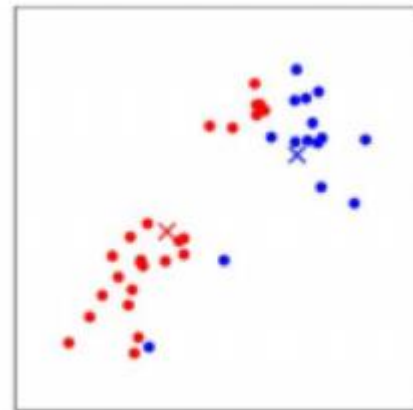
(a)



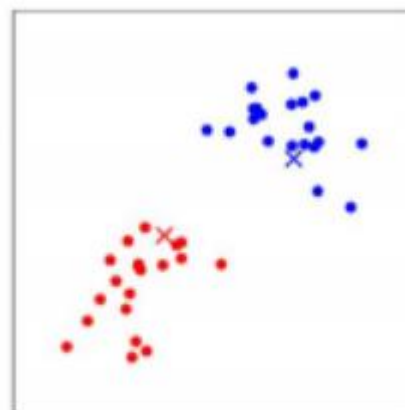
(b)



(c)



(d)



(e)

Example: K-Means

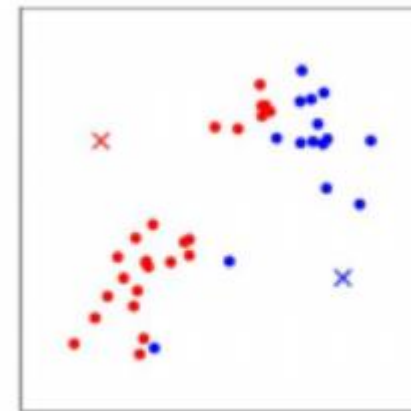
- Stopping point: stop when the group centers don't change much between iterations.



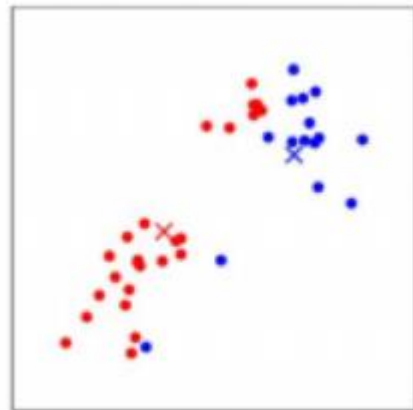
(a)



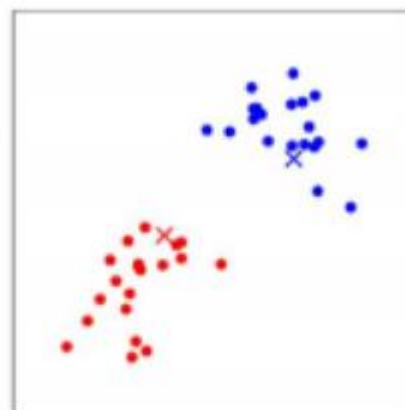
(b)



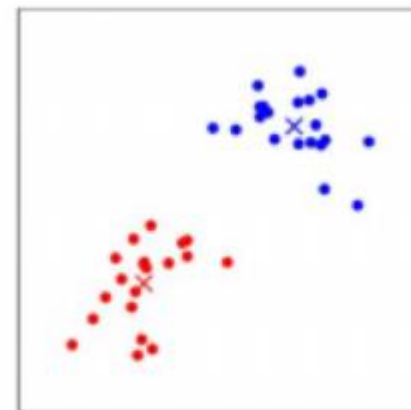
(c)



(d)



(e)



(f)

Mathematical Representation of K-Means

- **Objective:** to predict k centroids and a label for each datapoint.

1. Initialize **cluster centroids** $\mu_1, \mu_2, \dots, \mu_k \in \mathbb{R}^n$ randomly.

2. Repeat until convergence: {

For every i , set

$$c^{(i)} := \arg \min_j \|x^{(i)} - \mu_j\|^2.$$

For each j , set

$$\mu_j := \frac{\sum_{i=1}^m 1\{c^{(i)} = j\} x^{(i)}}{\sum_{i=1}^m 1\{c^{(i)} = j\}}.$$

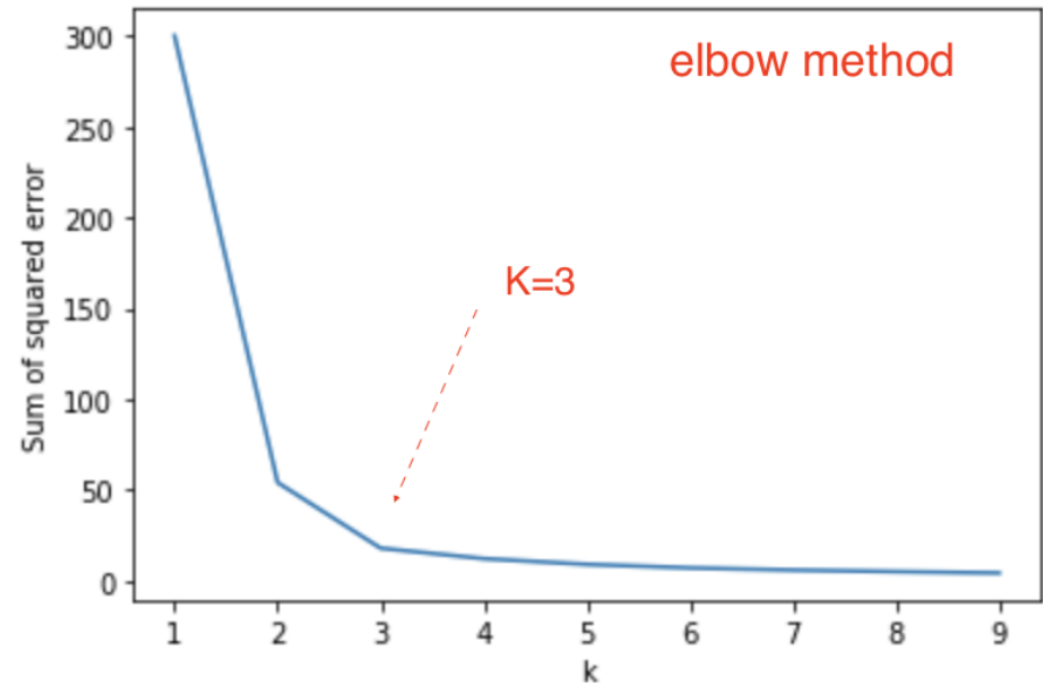
}

Evaluation Methods for K-Means

- Elbow method
- Silhouette analysis

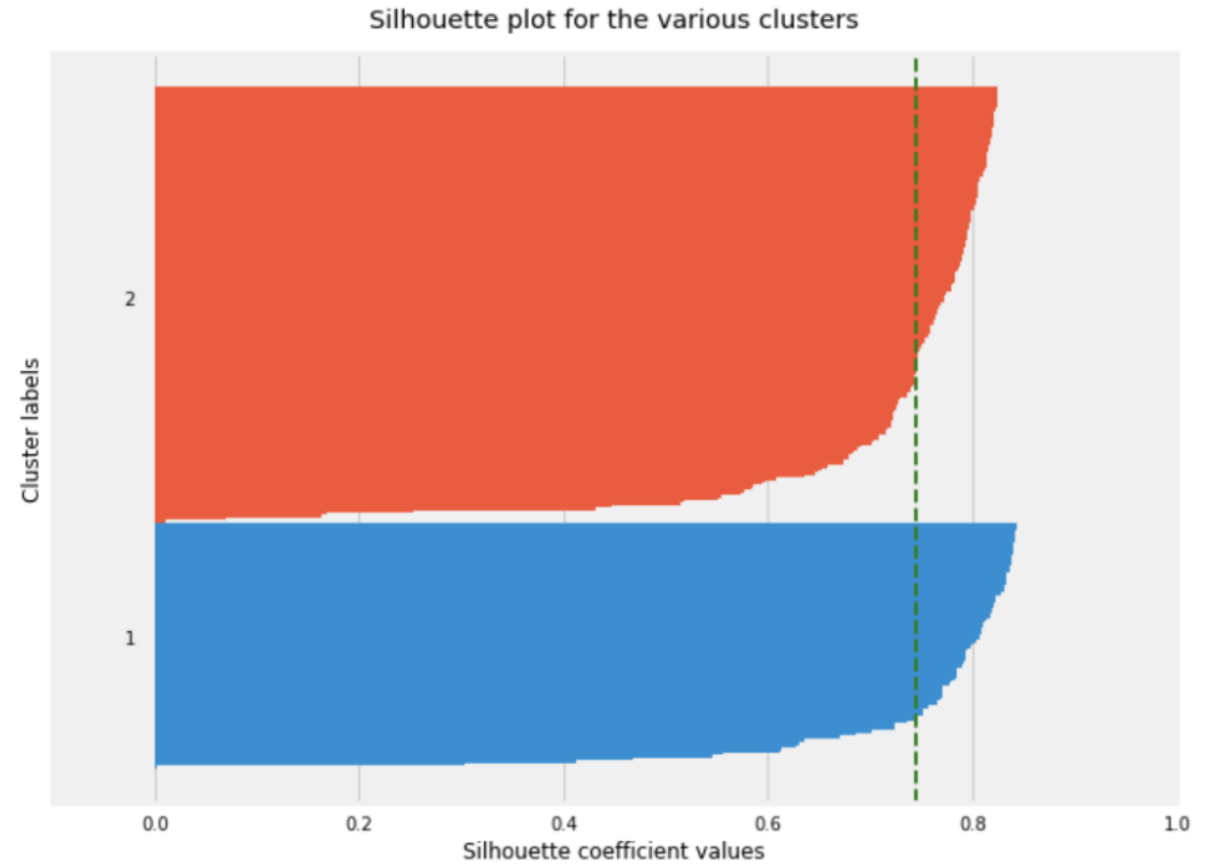
Evaluation Methods

- **Elbow method** uses the sum of squared distance (SSE) between data points and their assigned centroids.



Evaluation Methods

- Silhouette method



Evaluation Methods

- The silhouette coefficient is calculated as:

$$s(\mathbf{o}) = \frac{b(\mathbf{o}) - a(\mathbf{o})}{\max\{a(\mathbf{o}), b(\mathbf{o})\}}$$

- **$s(\mathbf{o})$** : the silhouette coefficient of the data point $\mathbf{o}$
- **$a(\mathbf{o})$** : the *average distance* between $\mathbf{o}$ and all the other data points in the cluster to which $\mathbf{o}$ belongs
- **$b(\mathbf{o})$** : the *minimum average distance* from $\mathbf{o}$ to all clusters to which $\mathbf{o}$ does not belong

Evaluation Methods

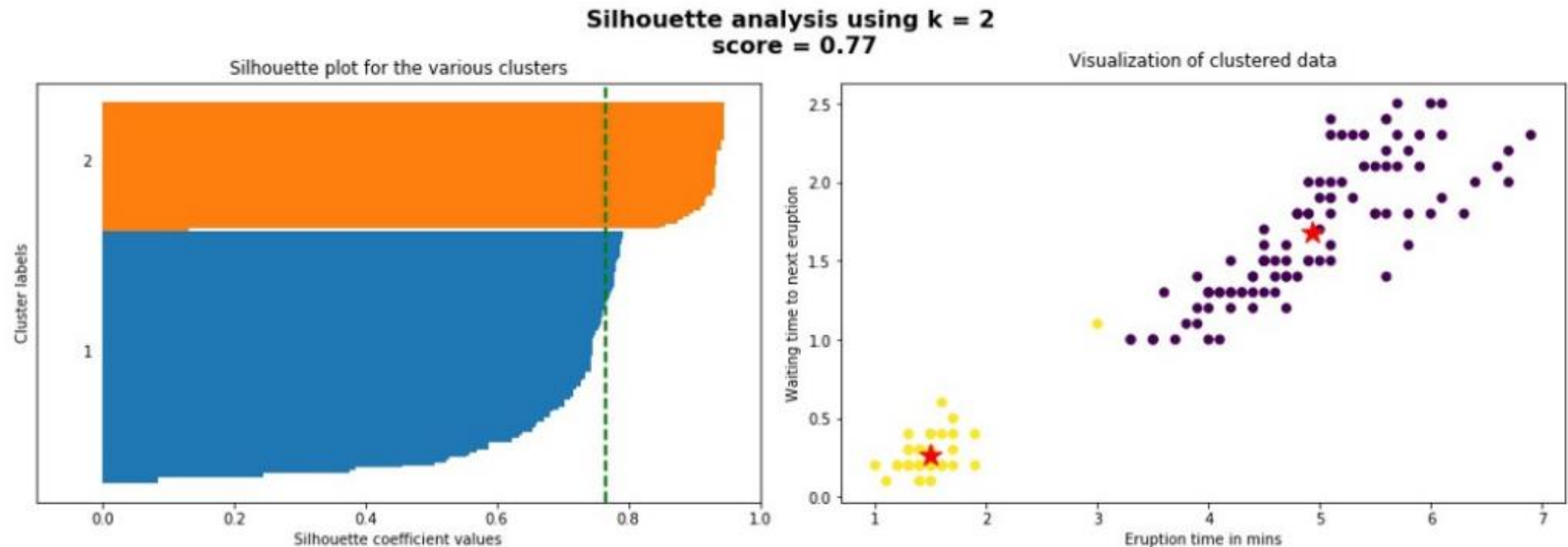
- Points about the silhouette coefficient :

$$s(\mathbf{o}) = \frac{b(\mathbf{o}) - a(\mathbf{o})}{\max\{a(\mathbf{o}), b(\mathbf{o})\}}$$

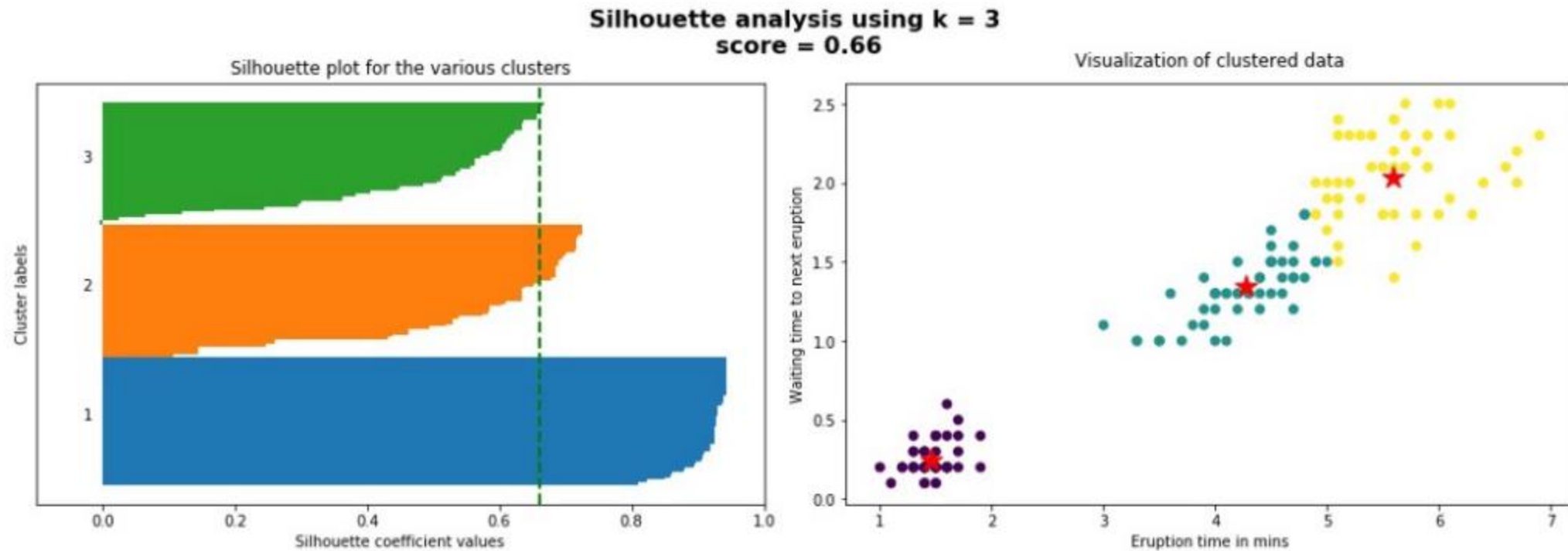
- The silhouette coefficient is between $[-1, 1]$.
- A score of **1** indicates the **best case** where the data point $\mathbf{o}$ is very compact within the cluster to which it belongs and far away from the other clusters.
- A score of **-1** shows the **worst** case.
- Values near **0** denote **overlapping** clusters.

Example of silhouette coefficient

- Most silhouette coefficients are above 0.8 when $k=2$:

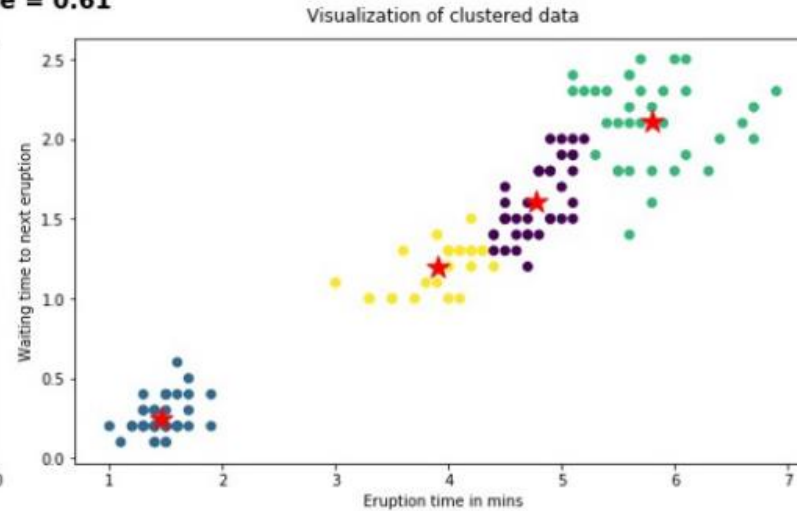
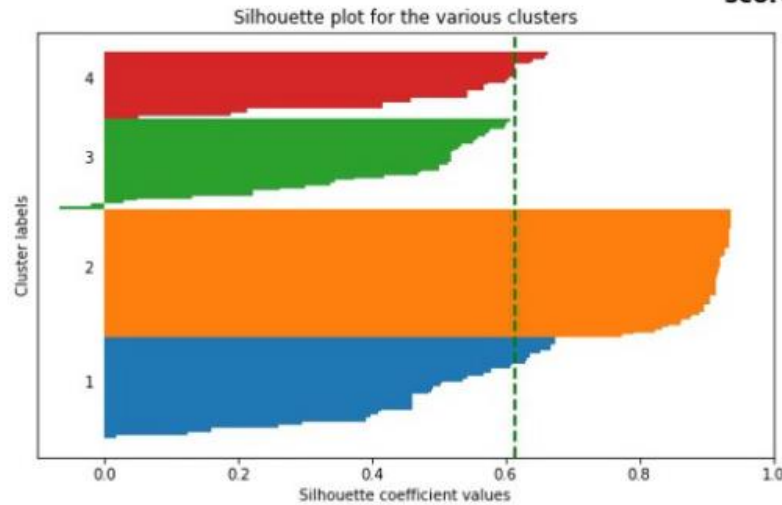


Example of silhouette coefficient

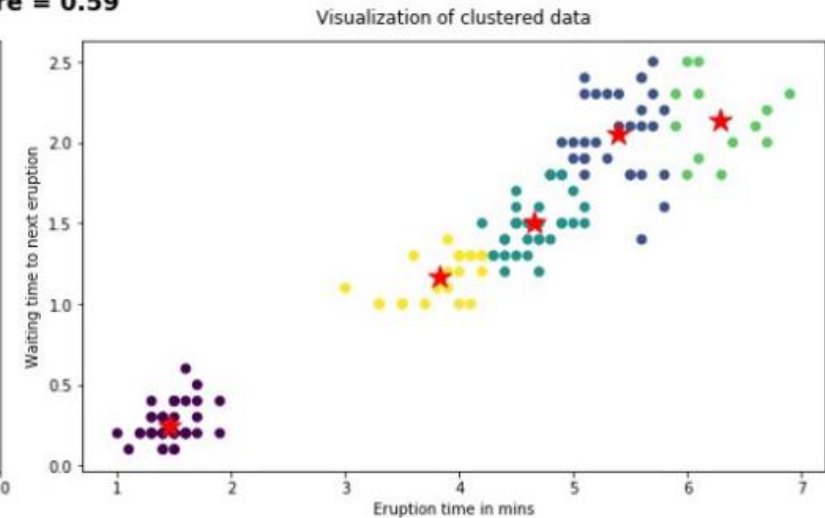
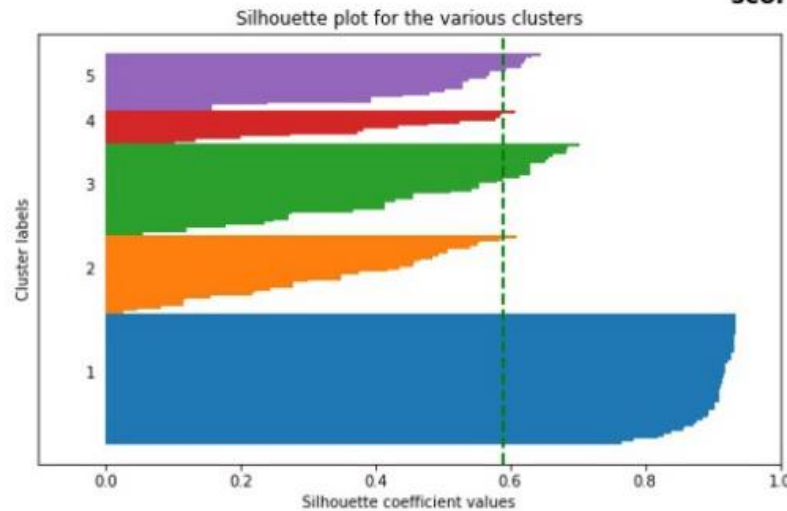


Example of silhouette coefficient

Silhouette analysis using k = 4
score = 0.61



Silhouette analysis using k = 5
score = 0.59

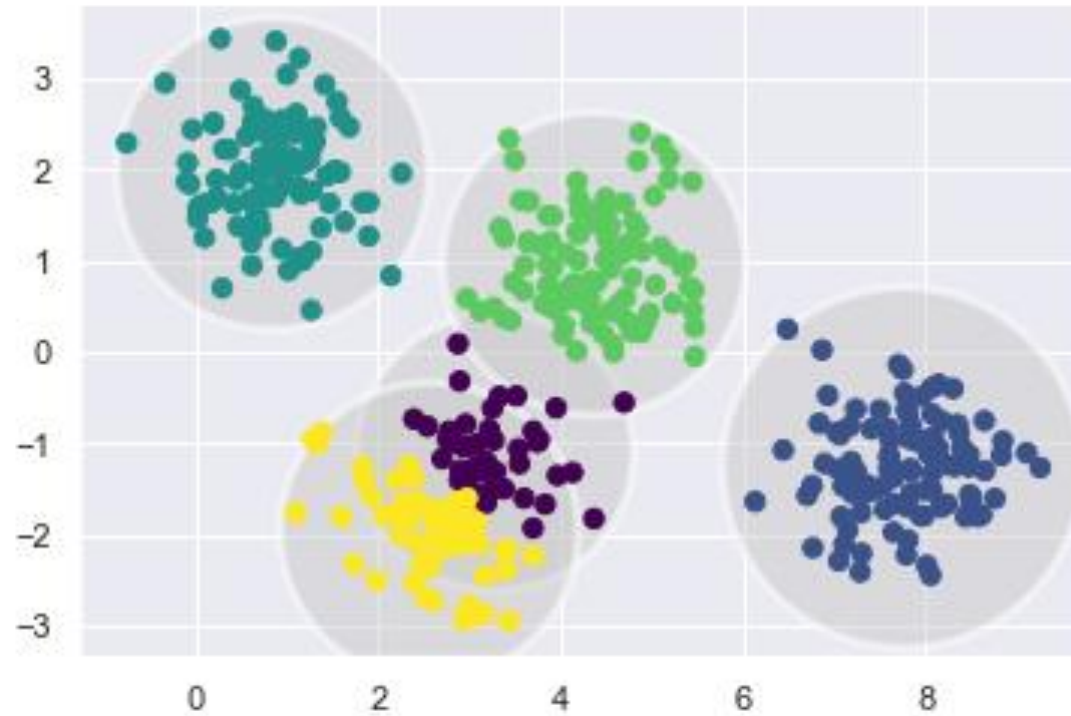


Disadvantages of K-Means

- The number of groups/classes (K) should be selected.
- It starts with a random choice of cluster centers, so it may result into different clustering outcomes on different runs of the algorithm (the results are not be repeatable, low consistency)
- Makes hard assignments of points to cluster.
- Sensitive to outliers.
- **K-Medians:** instead of recomputing the center points using the mean, the median is used (less sensitive to outliers but requires more computation, not efficient for larger datasets).

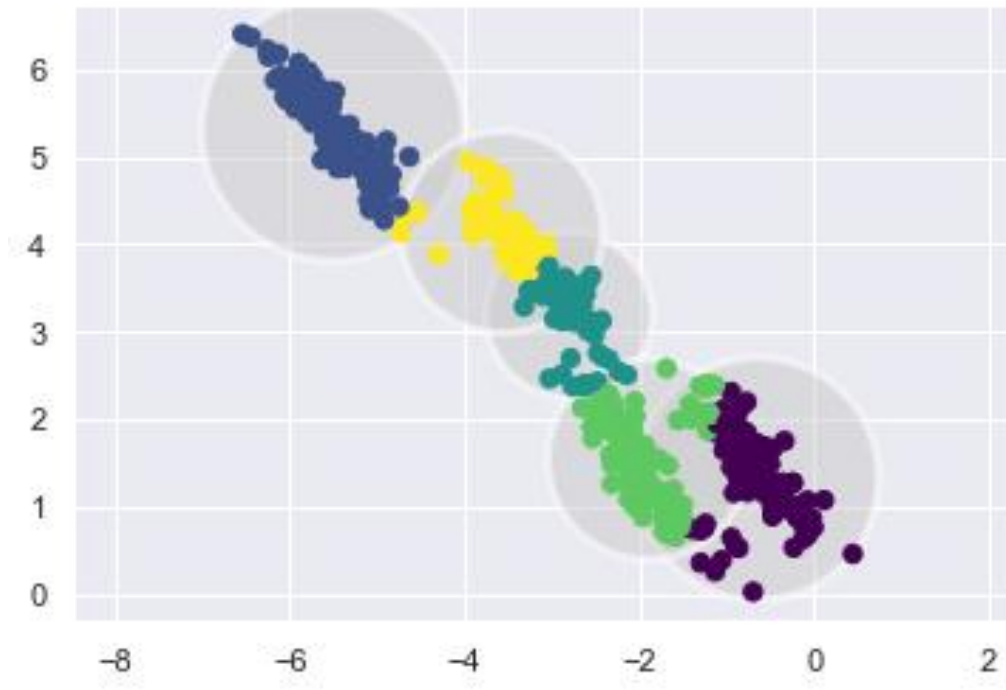
Weaknesses of K-means

- In K-means the cluster models must be circular



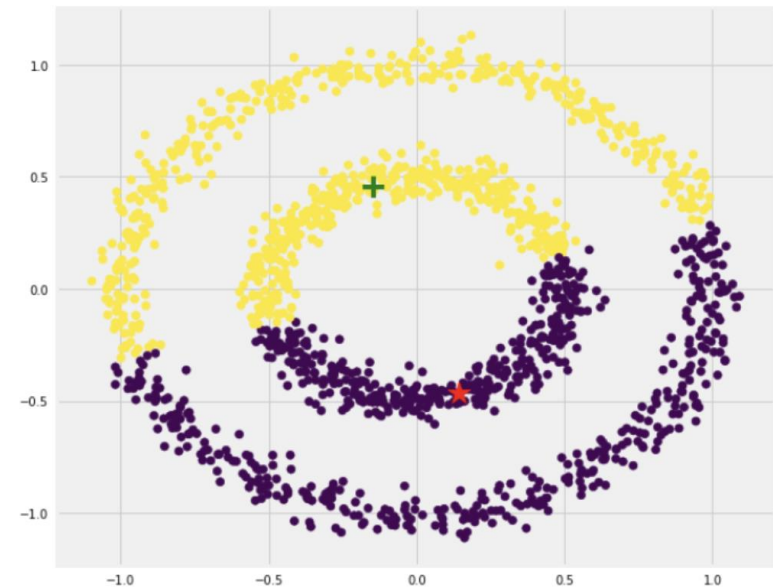
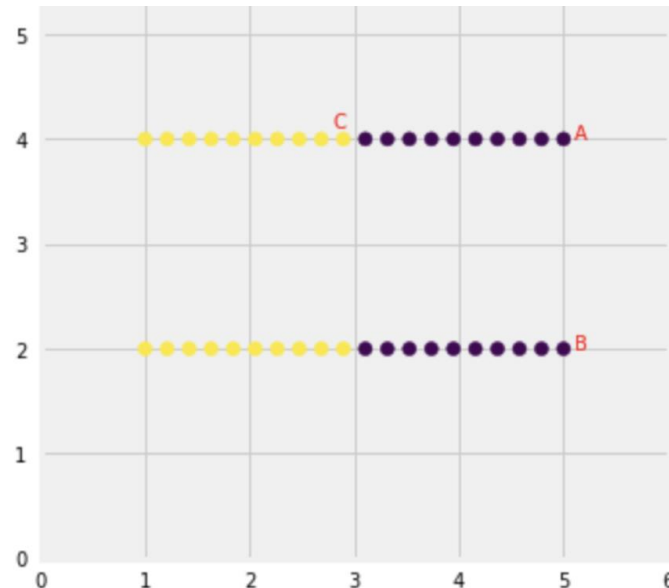
Let's transform the data

- Even visually we can see that a circular cluster is a poor fit.



More examples of k-means poor performance

- k-means can be viewed as a greedy algorithm since it tries to minimize the sum of the squared distances to the cluster centers.
- Poor job on non-convex shapes.

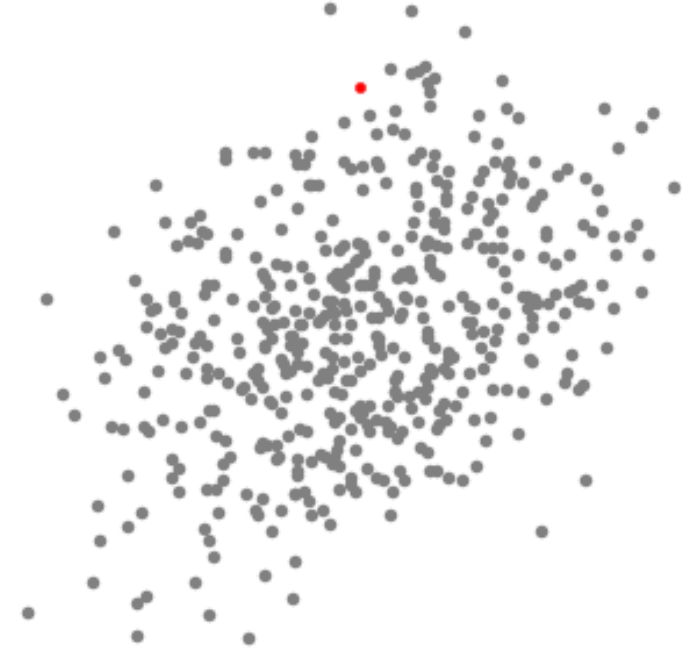


Clustering Algorithms

- K-Means Clustering
- **Mean-Shift Clustering**
- Density-Based Spatial Clustering of Applications with Noise (DBSCAN)
- Expectation-Maximization (EM) Clustering using Gaussian Mixture Models (GMM)
- Agglomerative Hierarchical Clustering

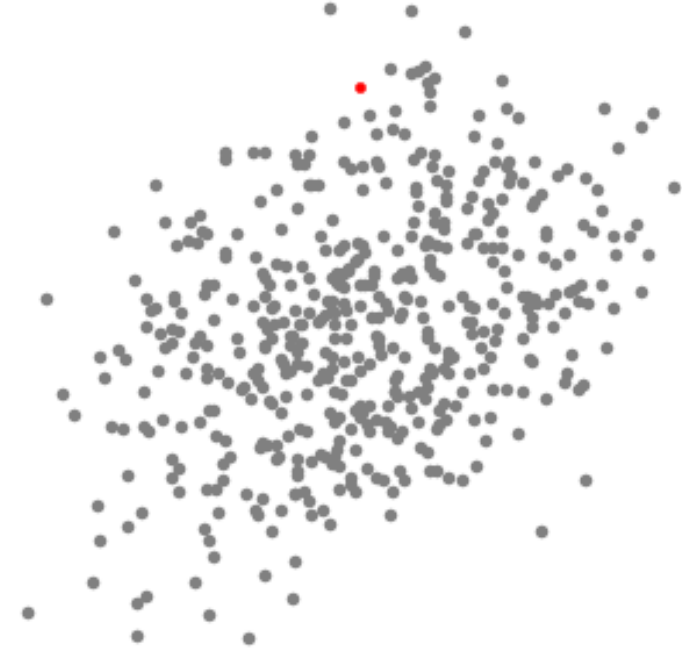
Mean-shift Clustering

- It is a sliding-window-based algorithm that attempts to find dense areas of data points.
- **Goal:** to locate the center points of each group/class (a centroid-based algorithm), which works by updating candidates for center points to be the mean of the points within the sliding-window.



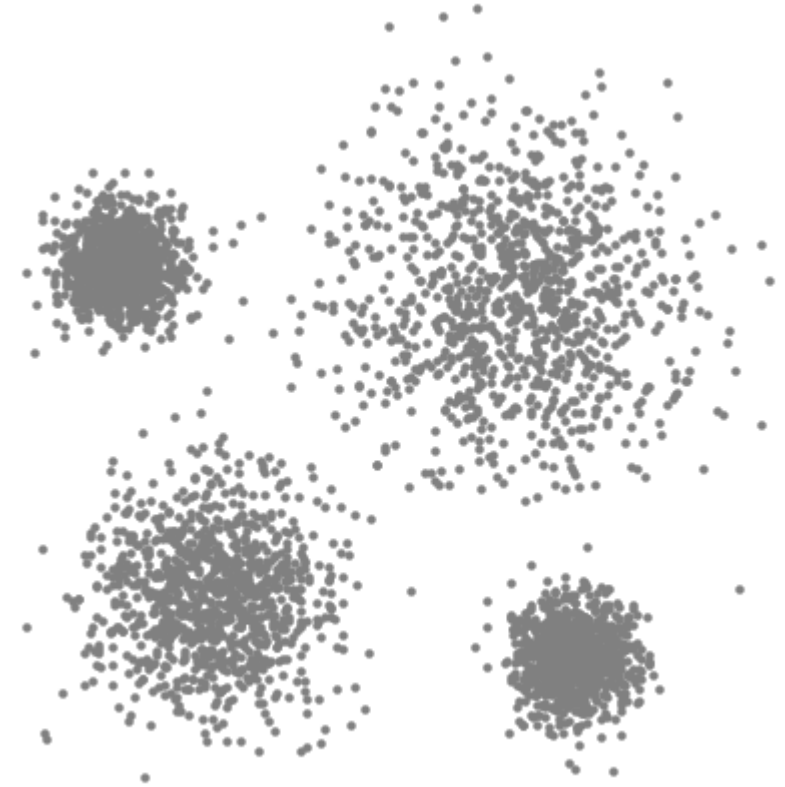
Mean-shift Clustering Steps

- **Step 1:** Randomly select a point C and begin with a circular sliding window centered at C and having radius r as the kernel.
- **Step 2:** At every iteration, the sliding window is shifted towards regions of higher density (this is a hill-climbing algorithm). This is done by shifting the center point to the mean of the points within the window.
- **Step 3:** Continue shifting the sliding window until there is no direction at which a shift can accommodate more points inside the kernel.



Mean-shift Clustering Steps

- Steps 1 to 3 is done with many sliding windows until all points lie within a window.
- When multiple sliding windows overlap, the window containing the most points is kept.
- The data points are then clustered according to the sliding window in which they reside.



Black dot: the centroid of a sliding window
Gray dot: a data point.



Mathematical Representation of Mean-shift Clustering

- **Recall:** Density Estimation discussion
- **Problem:** given discrete data sampled from a density function, how to locate the maxima (the modes) of a density function.
- Mean shift is an iterative process to find mode (**mode-seeking algorithm**).
- It starts with an initial estimate x . Let a kernel function be given.

$$K(x_i - x)$$

- This function determines the weight of nearby points for re-estimation of the mean. Typically, a Gaussian kernel on the distance to the current estimate is used:

$$K(x_i - x) = e^{-c\|x_i - x\|^2}$$

- The weighted mean of the density in the window determined by K is:

$$m(x) = \frac{\sum_{x_i \in N(x)} K(x_i - x)x_i}{\sum_{x_i \in N(x)} K(x_i - x)}$$

Mathematical Representation of Mean-shift Clustering

- The two most common kernels for mean shift:

- **Flat Kernel**

$$k(x) = \begin{cases} 1 & \text{if } x \leq \lambda \\ 0 & \text{if } x > \lambda \end{cases}$$

- **Gaussian Kernel:**

$$k(x) = e^{-\frac{x}{2\sigma^2}}$$

- At every iteration, the kernel is shifted to the centroid (the mean of the points within it).
- The method of calculating this mean depends on the choice of the kernel.
- If a Gaussian kernel is chosen, then every point will first be assigned a weight which will decay exponentially as the distance from the kernel's center increases.

Pros and Cons of Mean-shift Clustering

- **Advantages:**

- No assumption similar to K-means or Gaussian mixture.
- Can be used for data with nonconvex shape.
- It only requires one parameter, bandwidth, which automatically determines the number of clusters.
- No issue of local minima as we had in K-means.
- Outliers do not influence the final outcome.

- **Disadvantages:**

- Computationally intensive
- No control on the number of clusters